

Refactoring and Reflection

Exam No. B024703

March 29, 2017

1 Introduction

This short report describes and reflects upon the work done in refactoring a small web application that facilitates the playing of a tabletop game.

Development work was done according to Agile principles, with small user stories grouped into sprints and then completed. These individual stories or issues were stored and tracked using GitHub's tool for this purpose. The detail of these is not unnecessarily repeated here. Instead, this report provides an overview of the process, direction to where this further technical detail is documented, and a critical reflection on how successful this project was. This is followed by a brief conclusion.

2 The Process

The process was outlined to some extent in the *Planning and Risks* (P+R) document. This section provides more detail as to how this was done on a technical level.

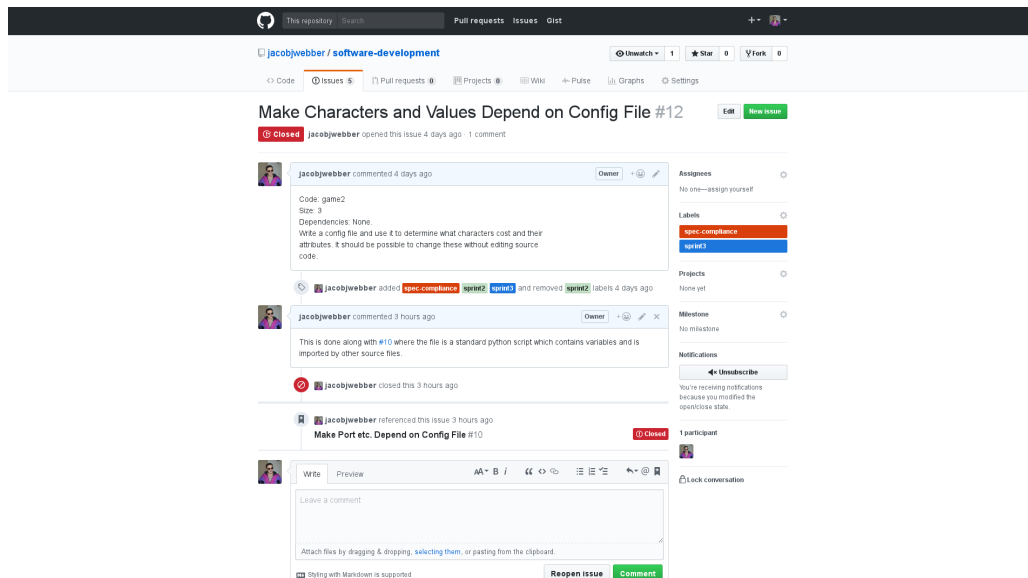


Figure 1: Screen-shot of the issue tracking in GitHub

2.1 Sprints and Sprint Planning

2.2 GitHub Issue Tracking

All of the issues discussed in P+R were uploaded to the GitHub issue tracking tool. These were marked according to their epic and according to what sprint they were in. Epics are particular genres of issues, such as bugs, and sprints were sections of work to be completed in one go. Only the issues that made up sprint 1 were determined in the P+R document, so new sprint planning sessions were done during the process. These sprint plans are also stored in the GitHub issue tracker. The issues in this form are very simple. They just allow a title and comments, as well as tags and the ability to be marked as open or closed. This was in general a positive, as the issues could be tagged with the appropriate sprint and epic, and marked as complete with no unnecessary complications. These issues can easily be searched for using these tags, so that the user can tell quickly, for example, how many issues are left in the current sprint. Figure 1 shows this clearly. Note how the issue is marked as closed, but it is easy to see what sprint it was resolved as. GitHub allows the easy referencing of other issues which is used here to show that two issues were resolved together. As well as allowing easy organisation, this method of doing work provides complete documentation of the development process that any future developer could look at. This can help explain design

decisions and contributes to a well documented source base with very little additional overhead.

These issues can be found at

<https://github.com/jacobjwebber/software-development/issues>

To see those issues that have been closed, it is necessary to clear the search bar. These issues should be taken as documentation of all the work that has been completed.

2.3 Git and Continuous Integration

As the name suggests, Git is at the very core of GitHub. Git is a revision control system that allows a developer to track changes, revert and form branches. These features were used extensively throughout the development process. The initial branching strategy was to leave the original untouched on branch “master” and to form a new branch for development work called “develop”. From this a new branch would be made for each issue tackled and then this would be merged back into “develop” when complete. However, this proved too time consuming, particularly when tackling smaller issues. Instead, only the very large issues followed this strategy, and all other work was done directly on branch “develop”. Preserving the original code as received on branch “master” proved a useful reference for later work. Only when development work was completed was this pulled into the “master” branch.

The use of GitHub and Git also allowed for continuous integration to be done with Travis CI. Travis CI is a product from GitHub that runs builds on a server. These builds perform the unit tests as defined in the build script every time code is committed to GitHub. The tool also provides a build status that can be embedded in the project’s home page, showing whether all tests are passing on each branch.

2.4 Debugging and Development

The Flask framework provides a built in debugger. This can be activated by setting the environment variable `FLASK_DEBUG` equal to 1. When there

is a error in the Python source the browser window will show a full stack trace in an easy-to-read format. Figure 2 shows what this looks like when a standard bug is hit. Flask also allows for automatic reloading, so that the server updates itself when source files are modified.

3 Reflection

4 Conclusion

Rename:

- 'Wizard' to 'Captain'
- 'Band' to 'Squad'
- 'Apprentice' to 'Ensign'

to more accurately reflect specification.

References

- [1] The Agile Alliance *TDD* <https://www.agilealliance.org/glossary/tdd/> Accessed: 01-02-2017
- [2] Scaled Agile *Spikes* <http://www.scaledagileframework.com/spikes/> Accessed: 01-02-2017

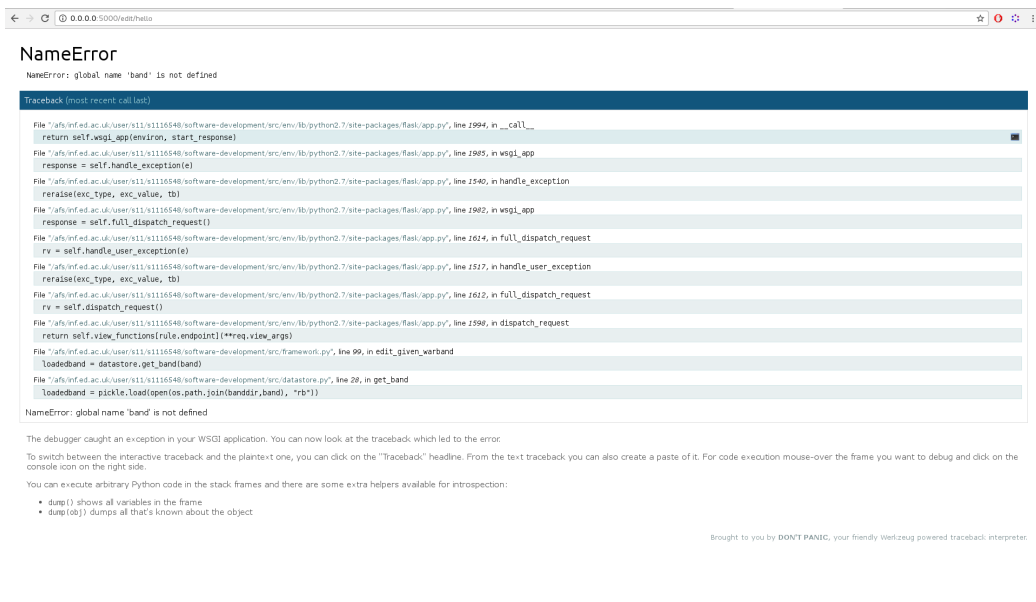


Figure 2: Screen-shot of the debugging process with Flask's built in debugger